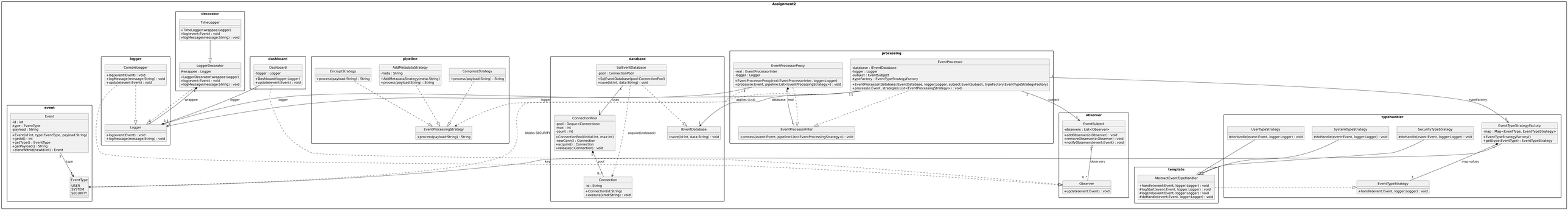
Assignment2
decorator
TimeLogger
+TimeLogger(wrappee:Logger)
+logEvent(Event) : void
+logMessage(message:String) : void
LoggerDecorator
#wrappee : Logger
+LoggerDecorator(wrappee:Logger)
+logEvent(Event) : void
+logMessage(message:String) : void
logger
ConsoleLogger
+log(event:Event) : void
+logMessage(message:String) : void
+update(event:Event) : void
Logger
+log(event:Event) : void
+logMessage(message:String) : void
dashboard
Dashboard
-logger : Logger
+Dashboard(logger:Logger)
+update(event:Event) : void
pipeline
EncryptStrategy
+process(payload:String) : String
AddMetadataStrategy
-meta : String
+AddMetadataStrategy(meta:String)
+process(payload:String) : String
CompressStrategy
+process(payload:String) : String
EventProcessingStrategy
+process(payload:String) : String
database
SqlEventDatabase
-pool : ConnectionPool
+SqlEventDatabase(pool:ConnectionPool)
+save(id:int, data:String) : void
ConnectionPool
-pool : Deque<Connection>
-max : int
-count : int
+ConnectionPool(initial:int, max:int)
-newConn() : Connection
+acquire() : Connection
+release(c:Connection) : void
Connection
-id : String
+Connection(id:String)
+execute(cmd:String) : void
IEventDatabase
+save(id:int, data:String) : void
processing
EventProcessorProxy
-real : EventProcessorInter
-logger : Logger
+EventProcessorProxy(real:EventProcessorInter, logger:Logger)
+process(e:Event, pipeline:List<EventProcessingStrategy>) : void
EventProcessor
-database : IEventDatabase
-logger : Logger
-subject : EventSubject
-typeFactory : EventTypeStrategyFactory
+EventProcessor(database:IEventDatabase, logger:Logger, subject:EventSubject, typeFactory:EventTypeStrategyFactory)
+process(e:Event, strategies:List<EventProcessingStrategy>) : void
EventProcessorInter
+process(event:Event, pipeline:List<EventProcessingStrategy>) : void
observer
EventSubject
-observers : List<Observer>
+addObserver(o:Observer) : void
+removeObserver(o:Observer) : void
+notifyObservers(event:Event) : void
Observer
+update(event:Event) : void
typehandler
UserTypeStrategy
#doHandle(event:Event, logger:Logger) : void
SystemTypeStrategy
#doHandle(event:Event, logger:Logger) : void
SecurityTypeStrategy
#doHandle(event:Event, logger:Logger) : void
EventTypeStrategyFactory
-map : Map<EventType, EventTypeStrategy>
+EventTypeStrategyFactory()
+getType(eventType) : EventTypeStrategy
template
AbstractEventTypeHandler
+handle(event:Event, logger:Logger) : void
#logStart(event:Event, logger:Logger) : void
#logEnd(event:Event, logger:Logger) : void
#doHandle(event:Event, logger:Logger) : void
EventTypeStrategy
+handle(event:Event, logger:Logger) : void
event
Event
-id : int
-type : EventType
-payload : String
+Event(id:int, type:EventType, payload:String)
+getId() : int
+getType() : EventType
+getPayload() : String
+cloneWithId(newId:int) : Event
EventType
USER
SYSTEM
SECURITY
wrappee
logger
logger
logger
logger
logger
uses
applies (List)
database
real
subject
typeFactory
observers
map values
blocks SECURITY
acquire()/release()
pool
key
type